

Memory in Toy Transformer Models

1 Introduction

The goal of this project is to understand how memorised facts are stored in the weights of transformer models. To isolate the mechanisms involved, we study a minimal *toy model*: a single-layer transformer that receives two input tokens and must produce one output token. A *fact* is defined as a specific pair of input tokens mapped to a specific output token; the mapping is random, so the model cannot exploit any systematic structure and must genuinely memorise each fact.

By selectively enabling or disabling components of the architecture—attention, feed-forward layers, biases, normalisation, residual connections—we can measure how many facts each configuration can learn as a function of its dimensions. In addition, we attempt to match the trained model’s performance with a hand-coded weight-setting algorithm (i.e. not learned via gradient descent) to gain further insight into the encoding.

2 Model Architecture

The model, MEMORYTOYMODEL, is a single-layer transformer whose components can be toggled on or off via a set of architectural flags. All configurable options are collected in a MODELSETTINGS object; the full list is summarised in Table 1 and described below.

2.1 Overview

The forward pass proceeds through three stages:

1. **Embedding.** The input tokens are mapped into a d_{residual} -dimensional residual stream. The precise mechanism depends on whether attention is enabled (see Section 2.2).
2. **Feed-forward block** (optional). A two-layer MLP transforms the residual-stream vector (see Section 2.4).
3. **Output head.** A final linear projection maps the residual stream to logits over the output vocabulary.

2.2 Embedding and Attention

Two modes are available, controlled by the `attention` flag.

With attention (`attention = True`). Each input token is embedded via a learned token embedding $W_E \in \mathbb{R}^{V_{\text{in}} \times d_{\text{residual}}}$ and summed with a learned positional embedding $W_P \in \mathbb{R}^{T \times d_{\text{residual}}}$, where T is the sequence length (typically 2) and V_{in} is the input vocabulary size. The resulting sequence is passed through an optional RMSNorm and then through a single causal self-attention layer with n_{heads} heads (Section 2.3). A residual connection adds the attention output back to

the embedding. Only the representation at the *last* sequence position is retained for subsequent processing.

Without attention (`attention = False`). A separate embedding matrix $W_{E_i} \in \mathbb{R}^{V_{\text{in}} \times d_{\text{residual}}}$ is learned for each input position $i \in \{1, \dots, T\}$. The residual-stream vector is the element-wise sum of all position embeddings:

$$x = \sum_{i=1}^T W_{E_i}[\text{token}_i].$$

This removes the attention mechanism entirely, replacing it with a simple additive composition of per-position embeddings.

2.3 Causal Self-Attention

When attention is enabled, the model uses multi-head causal (masked) self-attention. A single linear projection produces queries, keys, and values (Q, K, V), which are split across n_{heads} heads, each of dimension $d_{\text{head}} = d_{\text{residual}}/n_{\text{heads}}$. Attention scores are computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{head}}}} + M\right)V,$$

where M is a causal mask that sets future positions to $-\infty$. The outputs of all heads are concatenated and projected back to d_{residual} dimensions via a learned linear map. Notably, no bias is used in the attention projections.

2.4 Feed-Forward Block

When enabled (`ff = True`), the feed-forward block is a standard two-layer MLP:

$$\text{FF}(x) = W_2 \sigma(W_1 x + b_1) + b_2,$$

where $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{residual}}}$, $W_2 \in \mathbb{R}^{d_{\text{residual}} \times d_{\text{ff}}}$, and σ is a configurable activation function (GELU by default; ReLU is also supported). Biases b_1, b_2 are included only when `bias = True`. The input to the MLP is optionally normalised with RMSNorm (controlled by `norms`).

A residual connection around the feed-forward block is controlled by the flag `ff_residual`:

$$x \leftarrow \begin{cases} x + \text{FF}(\text{Norm}(x)) & \text{if } \text{ff_residual} = \text{True}, \\ \text{FF}(\text{Norm}(x)) & \text{otherwise.} \end{cases}$$

2.5 Output Head

The residual-stream vector is passed through a final RMSNorm (if `norms = True`) and then a linear projection $W_H \in \mathbb{R}^{V_{\text{out}} \times d_{\text{residual}}}$ (with optional bias) to produce logits over the output vocabulary of size V_{out} .

2.6 Configuration Summary

3 Fact Generation

Facts are generated deterministically from the random seed. Each fact consists of `seq_len` input tokens drawn (without replacement at the tuple level) from the input vocabulary and a target token assigned cyclically from the output vocabulary ($\text{target}_i = i \bmod V_{\text{out}}$). This ensures a uniform distribution over output classes while the input-output mapping remains arbitrary, preventing the model from learning a simple rule.

Table 1: Model settings and their descriptions.

Parameter	Default	Description
<i>Data dimensions</i>		
<code>seq_len</code>	2	Number of input tokens per fact
<code>input_vocab_size</code>	32	Size of the input token vocabulary
<code>output_vocab_size</code>	16	Size of the output token vocabulary
<code>n_facts</code>	64	Number of facts to memorise
<code>seed</code>	42	Random seed for fact generation
<i>Internal dimensions</i>		
<code>d_residual</code>	16	Residual-stream (embedding) dimension
<code>n_heads</code>	1	Number of attention heads
<code>d_ff</code>	16	Hidden dimension of the feed-forward block
<i>Architectural toggles</i>		
<code>attention</code>	True	Enable causal self-attention
<code>ff</code>	True	Enable feed-forward block
<code>bias</code>	True	Include bias terms in linear layers
<code>norms</code>	True	Use RMSNorm (otherwise identity)
<code>ff_residual</code>	True	Residual connection around FF block
<code>ff_activation_type</code>	GELU	Activation function (GELU or ReLU)

4 Training

The model is trained with the Adam optimiser, using binary cross-entropy with logits as the loss function (applied to one-hot encoded targets). Two accuracy metrics are tracked:

- **Accuracy:** the fraction of (fact, output-class) pairs for which the sign of the logit matches the one-hot label.
- **Best-guess accuracy:** the fraction of facts for which $\arg \max$ of the logits equals the correct target.

Training supports optional early stopping: it halts when perfect monitored accuracy is reached, or when the monitored accuracy has not improved for a specified number of patience epochs.

5 Experiment: Which architectural components matter for fact memorisation?

The results of this experiment are shown in Table 2, which reports the maximum number of facts successfully memorised for each architectural configuration. Outliers are highlighted with a `box` and the highest value in each row is shown in **bold**.

The results comes from four diffrent runs: `experiment_log_3att_duplicate`, `experiment_log_3att`, `experiment_log`, and `E1/experiment_log`. For the last one only results with attentions are included becasue of a bugg in the code for non-attention cases, during that run.

5.1 Comments on outliers

Outliers are highlighted with a `box`. Almost all outliers are low values, which indicates that they are due to training failures. The one exception is the high outlier of 768 facts memorised for the configuration with Attention, FF, Norms, Residual, Bias, and ReLU activation. I don't now

what’s up with that, but I think it indicates that higher number of facts is generally possible, with a bit of luck, and/or longer training.

There are lots of outliers in the experiments which did not run multiple attempts for each number of facts. This shows that running multiple attempts is important for avoiding being misled by training failures.

5.2 Attention and FF obviously matters

Including Attention or not, and including a Feed-Forward block or not, obviously matters, which is also confirmed in table 2.

Notice that excluding Attention leads to more facts memorised. This is not surprising. The reason is because the attention replacement is more powerful than the attention itself.

Next we’ll go over the other settings one by one, in order of least expected importance.

5.3 Does GELU vs ReLU matter? - Probably not much

I don’t think GELU vs ReLU has a significant impact on the maximum number of facts that can be learned. The main reason for this conclusion is that neither GELU nor ReLU consistently outperform the other option, across the different architectural configurations.

Looking at table 2, we can see that the choice of activation function (GELU vs ReLU) does not have a consistent impact on the maximum number of facts memorised across different architectural configurations. Comparing GELU vs ReLU for each combination of the other settings, we find that **GELU have higher Max Facts in 9 cases, ReLU have higher Max Facts in 5 cases, and they score equally in 2 cases.**

Most of the time the difference in Max Facts is small, except for

- Attention: ✓, FF: ✓, Norms: ✓, Residual: ✓, Bias: ✓
where the difference is 576 vs 768, in favour of ReLU.
- Attention: ✓, FF: ✓, Norms: ✓, Residual: ✗, Bias: ✗
where the difference is 528 vs 384 in favour of GELU.
- Attention: ✓, FF: ✓, Norms: ✗, Residual: ✓, Bias: ✓
where the difference is 304 vs 432 in favour of ReLU.

I think these are due to training instabilities rather than a significant fact learning capacity between ReLU and GELU.

5.4 Does including a Bias matter - Probably not much

Including Bias or not in the feed-forward block does not seem to have a bit more impact than the choice of activation function. Bias mostly does better than No Bias, but there are some counter examples.

Looking at table 2, we can see that the inclusion of a Bias in the FF module, does not have a consistent impact on the maximum number of facts memorised across different architectural configurations. Comparing Bias vs No Bias for each combination of the other settings, we find that **Bias have higher Max Facts in 11 cases, No Bias have higher Max Facts in 3 cases, and they score equally in 1 case.**

The largest differences in Max Facts between Bias and No Bias are observed in the following configurations:

- Attention: ✓, FF: ✓, Norms: ✓, Residual: ✓, Activation: ReLU
where the difference is 768 vs 536 in favour of Bias.
- Attention: ✗, FF: ✓, Norms: ✓, Residual: ✗, Activation: GELU
where the difference is 792 vs 584 in favour of Bias.
- Attention: ✗, FF: ✓, Norms: ✓, Residual: ✗, Activation: ReLU
where the difference is 704 vs 512 in favour of Bias.

There are some effects of including the Bias. The effect size is bigger than for GELU vs ReLU. But this effect should decrease as the network scales, so going forward, I will chose not to worry about this.

5.5 Does including a Residual connection matter? - Yes

Looking at table 2, we can see that the inclusion of a Residual connection in parallel to the FF module, has a mostly consistent impact on the maximum number of facts memorised across different architectural configurations. Comparing Residual vs No Residual for each combination of the other settings, we find that **Residual have higher Max Facts in 15 cases, No Residual have higher Max Facts in 1 case.**

The one case where No Residual outperforms Residual is:

- Attention: ✓, FF: ✓, Norms: ✗, Bias: ✓, Activation: GELU
where the difference is 308 vs 384 in favour of No Residual.

For the rest of the configurantions, Residual outperforms no Residual with 5.1% - 50.0% (25.9% average) more Facts memorised.

5.6 Does including a Layer Norms matter? - Yes

Looking at table 2, we can see that the inclusion of a Layer Norms, has a mostly consistent impact on the maximum number of facts memorised across different architectural configurations. Comparing Layer Norms vs No Layer Norms for each combination of the other settings, we find that **Layer Norms have higher Max Facts in 17 cases, No Layer Norms have higher Max Facts in 1 case.**

The one case where No Layer Norms outperforms Layer Norms is:

- Attention: ✗, FF: ✓, Norms: ✗, Bias: ✗, Activation: GELU
where the difference is 512 vs 552 in favour of No Layer Norms.

The inclusion of Layer Norm has a much large inpact on when there is no FF module. Excluding the two no FF cases, and the configuration mentioned above, Layer Norms outperforms no Layer Norms with 4.7% - 89.5% (32.0% average) more Facts memorised.

When there is no FF module, Layer Norms outperforms no Layer Norms with 300.0% - 433.3% (366.7% average) more Facts memorised.

Table 2: Maximum facts memorised per architectural configuration ($d_{\text{residual}} = 16$, $d_{\text{ff}} = 16$). Columns Dup, 3att, and Orig correspond to `experiment_log_3att_duplicate`, `experiment_log_3att`, and `experiment_log`, respectively. E1 corresponds to `E1/experiment_log` (attention-only configurations). The highest value in each row is shown in **bold**.

Attention	FF	Norms	Residual	Bias	Activation	Max Facts			
						Dup	3att	Orig	E1
✓	✓	✓	✓	✓	GELU	568	576	248	528
✓	✓	✓	✓	✓	ReLU	536	536	768	368
✓	✓	✓	✓	✗	GELU	560	576	576	512
✓	✓	✓	✓	✗	ReLU	536	504	496	512
✓	✓	✓	✗	✓	GELU	504	504	464	520
✓	✓	✓	✗	✓	ReLU	456	512	432	56
✓	✓	✓	✗	✗	GELU	528	528	120	480
✓	✓	✓	✗	✗	ReLU	368	384	288	320
✓	✓	✗	✓	✓	GELU	304	264	48	80
✓	✓	✗	✓	✓	ReLU	432	304	160	160
✓	✓	✗	✓	✗	GELU	368	424	152	288
✓	✓	✗	✓	✗	ReLU	416	512	96	120
✓	✓	✗	✗	✓	GELU	384	232	304	256
✓	✓	✗	✗	✓	ReLU	248	368	320	296
✓	✓	✗	✗	✗	GELU	280	240	224	336
✓	✓	✗	✗	✗	ReLU	320	272	352	8
✓	✗	✓	N/A	N/A	N/A	256	256	200	24
✓	✗	✗	N/A	N/A	N/A	48	48	24	32
✗	✓	✓	✓	✓	GELU	832	792	808	
✗	✓	✓	✓	✓	ReLU	808	816	832	
✗	✓	✓	✓	✗	GELU	744	744	728	
✗	✓	✓	✓	✗	ReLU	744	736	728	
✗	✓	✓	✗	✓	GELU	792	768	768	
✗	✓	✓	✗	✓	ReLU	704	632	680	
✗	✓	✓	✗	✗	GELU	576	584	568	
✗	✓	✓	✗	✗	ReLU	480	512	248	
✗	✓	✗	✓	✓	GELU	688	696	680	
✗	✓	✗	✓	✓	ReLU	672	688	672	
✗	✓	✗	✓	✗	GELU	648	648	688	
✗	✓	✗	✓	✗	ReLU	672	672	672	
✗	✓	✗	✗	✓	GELU	536	560	576	
✗	✓	✗	✗	✓	ReLU	552	544	544	
✗	✓	✗	✗	✗	GELU	536	544	536	
✗	✓	✗	✗	✗	ReLU	544	552	504	
✗	✗	✓	N/A	N/A	N/A	240	248	256	
✗	✗	✗	N/A	N/A	N/A	64	64	64	